WATTWISE

Full-Stack + AI Project Roadmap

A guided build-and-learn plan for an electricity consumption analysis app designed around real Indian household data.

Project goal
Build a working application that accepts monthly electricity bills and manual meter readings, reconciles the two, identifies unusual consumption patterns, forecasts consumption/bills when enough data exists, and explains insights using evidence rather than guesses.

Approach: learn the unfamiliar Python/AI parts as you build. Your full-stack skills are the foundation; the AI layer comes after trustworthy data and calculations.

Project guide • Editable tracker included • No fixed two-week deadline

1. Product definition and scope

WattWise is not just a chatbot over electricity bills. The core product is a reliable data and analytics system. AI adds value after the app can calculate usage correctly and show where every insight came from.

Two supported data-entry modes

- **Option A — Monthly Bill Mode:** enter bill period, opening and closing meter readings when available, billed units, bill amount, and whether the reading is actual or estimated. Bill values remain the billing reference.
- **Option B — Manual Meter Mode:** record a cumulative meter reading daily or weekly, with timestamp and optional note/photo later. Usage for an interval is the new reading minus the previous valid reading.
- **Reconciliation:** when manual readings cover the same dates as a bill, compare the interval total with billed units. Show a discrepancy for review; never silently overwrite or blend values.

Core user outcomes

- See monthly units, cost, and trends in one dashboard.
- Spot unusual changes against a clearly stated baseline.
- Get a forecast only when data volume and quality support it, with uncertainty and limitations shown.
- Receive explanations tied to measured facts (for example, “usage was 24% above your last three complete periods”), not unsupported appliance-level claims.
- Record an action and later compare outcomes, without promising guaranteed savings.

Not in the first release

- Smart-meter integrations or custom hardware; appliance-level detection from aggregate readings; payment features; mobile app; complex multi-tenant permissions; a chatbot that invents causes. These can be revisited only after the core system works.

Important data rule

Monthly bills are usually monthly/periodic snapshots, not continuous sensor data. Manual cumulative readings are useful only when timestamps and meter identity are reliable. Different date ranges, estimated bills, meter replacements/resets, duplicate readings, and missing intervals must be represented explicitly.

2. Recommended technology stack

Layer	Use	Why / what to learn
Frontend	React + Vite, Tailwind CSS, shadcn/ui, React Router, Recharts	Build forms, validation states, dashboard charts, loading/error states, and responsive layouts. Use familiar React skills first.
Main API	Node.js + Express	Own REST API, validation, authentication later, business rules, and orchestration. Keep calculations deterministic.
Database	PostgreSQL + pg	Relational schema, foreign keys, constraints, indexes, transactions, date-range queries, migrations/seeding.
Python analytics	Python 3, venv, pandas, NumPy, scikit-learn, pytest	Start with Python basics and pandas; write tested analysis functions before training ML models.
Python service	FastAPI + Pydantic	Expose analytics/forecast endpoints as a small separate service once local Python functions work.
Charts	Recharts	Show actual consumption, comparisons, bill periods, forecast bands, and anomaly markers.
LLM layer	Provider API behind a small server-side adapter (choose provider later)	Turn verified metrics into plain-language explanations. Never use the LLM for arithmetic, database truth, or unsupported causal claims.
Quality & tooling	Git/GitHub, Postman or Bruno, ESLint, environment variables, logging	Repeatable development, API testing, safe secret handling, and debugging.

Architecture recommendation

Keep the React app → Express API → PostgreSQL path as the main system. Add a Python/FastAPI analytics service only after the calculation logic works in local Python tests. Express can call that service internally. Keep LLM calls on the server, with timeouts, cost limits, and a structured response schema.

Avoid for now

Do not introduce a vector database, agents, a complex microservice fleet, deep learning, or real-time streaming. They do not solve the first version’s biggest problem: clean, comparable consumption data.

3. Phased roadmap

Move through the phases in order. A phase is complete when its exit criteria are met—not just when the UI looks finished. You can spend as long as needed learning a topic before moving on.

Phase 0 — Product and repository setup

Goal: make the project easy to understand and change.

- Write the problem statement, user stories, and a small scope document.
- Create Git repository, README, issue board, .gitignore, and .env.example.
- Set up React/Vite frontend and Express backend with health endpoint.
- Choose consistent naming, API error format, and folder structure.

Deliverable: running frontend and API, clean repo, documented scope.

Phase 1 — Data model and data quality

Goal: represent bills and meter readings without losing context.

- Design users/properties/meters, bills, meter_readings, and analysis_runs (add auth tables only when needed).
- Store units, dates, source, actual/estimated status, meter identity, and period coverage.
- Add database constraints and validation for duplicate timestamps, negative/invalid readings, and meter resets.
- Create synthetic sample data plus several intentionally bad cases.

Exit check: data can be stored and invalid records are rejected or flagged.

Phase 2 — Full-stack entry flows

Goal: users can enter both data modes.

- Build Monthly Bill form and Manual Meter Reading form as separate flows.
- Add client and server validation; never trust frontend-only validation.
- Implement list/detail views, edit/correct workflow, and clear source labels.
- Add API tests for valid, invalid, duplicate, and boundary cases.

Exit check: a user can enter, view, correct, and delete/void test records safely.

Phase 3 — Deterministic calculations and reconciliation

Goal: make the numbers trustworthy before AI.

- Calculate interval usage = current cumulative reading – previous valid cumulative reading.
- Aggregate manual intervals only when they cover the requested bill period appropriately.
- Compare manual total with billed units; report delta and percentage difference.
- Handle gaps, overlaps, estimated bills, meter changes/resets, and incomplete coverage explicitly.

Exit check: known test cases produce expected totals and discrepancy flags.

Phase 4 — Dashboard and descriptive analytics

Goal: explain what happened, before predicting what happens next.

- Create summary cards: units, bill amount, average daily usage when valid, and comparison to previous period.
- Build time-series charts and bill-period filters.
- Add clear empty, incomplete-data, and error states.
- Use transparent baselines such as prior complete periods; label the comparison window.

Exit check: every metric can be traced to a query/formula and its date range.

Phase 5 — Python foundations (learn while building)

Goal: become comfortable writing and testing analysis code.

- Learn Python syntax, variables, collections, loops, functions, exceptions, modules, and virtual environments.
- Learn pandas Series/DataFrames, reading CSV, filtering, grouping, joins, date/time parsing, missing values, and plotting basics.
- Learn NumPy arrays and simple descriptive statistics; write pytest tests.
- Re-create a few existing consumption calculations in Python and compare them to Node results.

Exit check: you can load sample data, clean it, compute period summaries, and explain the code.

Phase 6 — Python analytics service

Goal: make tested Python functions callable from the app.

- Create a FastAPI service with Pydantic request/response models.
- Add endpoints for data-quality checks and period summaries first—not ML yet.
- Call it from Express over an internal URL; handle timeout/unavailable-service errors.
- Keep business-critical validation in the main API too; log request IDs and errors.

Exit check: API integration tests verify Node → Python → structured response.

Phase 7 — Anomaly detection

Goal: flag unusual usage with evidence and low false confidence.

- Begin with rules: minimum data checks, rolling median/mean, percentage change, and robust thresholds.
- Define what counts as an anomaly and show baseline/window/threshold in the UI.
- Evaluate on labelled synthetic scenarios: genuine spike, normal seasonal increase, missing data, and meter reset.
- Only later experiment with Isolation Forest or another scikit-learn method if enough representative data exists.

Exit check: anomalies are explainable, tested, and not emitted for inadequate data.

Phase 8 — Forecasting

Goal: estimate future usage and bill with honest uncertainty.

- Start with a naive baseline (for example, recent average daily usage scaled to the target period).
- Compare forecasts to held-out historical periods; track MAE or MAPE where appropriate.
- Consider seasonality and billing-period length; distinguish consumption forecast from bill forecast.
- Show a range or confidence caveat and do not forecast from too few/incomplete periods.

Exit check: a baseline beats or is meaningfully compared with a naive benchmark on test data.

Phase 9 — Evidence-grounded AI explanations

Goal: translate verified analysis into understandable language.

- Prepare a compact JSON facts object from the backend: values, date range, baseline, delta, quality flags.
- Ask the LLM to explain only supplied facts and to say when evidence is insufficient.
- Validate structured output; handle refusal, malformed responses, rate limits, and provider failure.
- Add a visible “why this insight?” evidence panel and avoid claims about specific appliances without supporting data.

Exit check: explanations match backend metrics and never replace the source calculations.

Phase 10 — User actions, evaluation, and polish

Goal: turn insights into a product loop.

- Let users mark an insight useful/not useful and record an action they tried.
- Compare later complete periods cautiously; note weather, occupancy, and tariff changes as confounders.
- Add automated tests, accessibility checks, responsive polish, logging, and seeded demo mode.
- Document known limitations, sample-data assumptions, and setup instructions.

Exit check: a reviewer can run the app locally and follow a complete demo journey.

Phase 11 — Deployment and portfolio packaging

Goal: produce a stable demo, not necessarily a public business launch.

- Choose hosting for frontend, Node API, Python service, and PostgreSQL.
- Set secrets in hosting environment; use migrations and a repeatable seed script.
- Add health checks, basic monitoring, database backup strategy, and a demo account/sample dataset.
- Create architecture diagram, screenshots, short demo script, and project write-up.

Exit check: clean setup instructions and repeatable end-to-end demo.

4. What to learn in Python and AI/ML

Topic	Learn only what WattWise needs	Practice task
Python basics	Types, lists/dicts, loops, functions, imports, exceptions, files, venv, pip	Write a function that calculates usage between two meter readings and rejects a negative delta.
pandas	DataFrames, datetime, sorting, filtering, groupby/resample, missing data, merges	Load meter_readings.csv and produce daily/monthly usage summaries.
Testing	pytest, fixtures, edge cases, deterministic functions	Test duplicate timestamps, reset meter, missing days, and a bill/manual match.
Statistics	Mean/median, variation, percent change, rolling windows, outliers, train/test split	Flag usage above a rolling median by a configurable threshold.
scikit-learn	Train/test data, feature preparation, baseline comparisons, Isolation Forest only if justified	Compare a rule-based detector with Isolation Forest on labelled synthetic cases.
Forecast evaluation	Naive baseline, MAE, MAPE caveats, time-based validation, leakage	Forecast the next period from prior periods and score on a held-out period.
LLM integration	Prompt constraints, structured output, JSON validation, timeout/retry, privacy/cost	Generate a short explanation from supplied facts and test that it cannot alter numeric values.

Recommended learning order

- Python fundamentals → pandas → tests and data cleaning.
- Descriptive statistics → transparent anomaly rules → evaluate on test cases.
- Forecasting baselines and time-based evaluation → only then consider more complex ML.
- LLM API basics and structured outputs after analytics results are already correct.

You do not need to become an ML researcher. The target is practical competence: understand the assumptions, write tests, evaluate the model against a baseline, and know when not to trust the output.

5. Data design and core rules

Entity	Important fields (starter set)	Purpose
Property	id, name/label, timezone, created_at	Represents a household/location; keep room for multiple properties later.
Meter	id, property_id, label, unit, active_from, active_to, status	Keeps readings associated with the correct physical meter.
Bill	id, property_id, meter_id, period_start, period_end, opening_reading, closing_reading, billed_units, amount, reading_type, source	Preserves original billing evidence and actual/estimated status.
MeterReading	id, meter_id, recorded_at, cumulative_value, unit, source, note, quality_status	Stores manual observations without pretending they are continuous sensor readings.
AnalysisRun	id, property_id, period_start, period_end, method_version, created_at, data_quality	Records what data/method produced an insight so results can be audited.
Insight/Feedback	id, analysis_run_id, type, facts_json, explanation, feedback, action_taken, created_at	Stores the evidence and user response to generated insights.

Rules to encode and test

- Use kWh consistently for energy; store currency amount separately and keep tariff details out of the first calculation unless needed.
- For cumulative readings, interval usage = later reading – earlier reading only when meter identity and chronology are valid.

- A negative delta may indicate a reset/replacement/data-entry issue; flag it rather than treating it as negative consumption.
- Do not compare a manual total with a bill unless the time windows align. If coverage is incomplete, label the comparison incomplete.
- Keep billed units, meter-reading delta, and calculated manual interval total as separate values.
- Use UTC or an explicit timezone consistently; dates are household-local billing dates and may not represent midnight readings.

6. Testing strategy and demo scenarios

Scenario	Expected behavior
Matching data	Bill opening 12450, closing 12750 → 300 kWh; complete same-period manual intervals also total 300 kWh → match.
Discrepancy	Bill says 300 kWh; manual same-period total says 335 kWh → show +35 kWh discrepancy and request review.
Incomplete readings	Missing intervals → do not claim full-period reconciliation; show coverage limitation.
Meter reset/replacement	Negative cumulative delta → flag reset/replacement or entry error; do not compute negative consumption.
Estimated bill	Preserve estimated status; show lower confidence for comparison until an actual bill/reading is available.
Duplicate timestamp	Reject or ask user to resolve duplicates for the same meter/time.
Normal seasonal increase	Avoid declaring a definitive anomaly from one simple threshold; show baseline and allow context/feedback.
Insufficient history	Explain that forecast/anomaly detection is unavailable or low confidence rather than fabricate a precise prediction.

7. API plan (build progressively)

Method / route	Purpose	Phase
GET /api/health	Check Express service is running.	0
POST /api/bills	Create a validated bill record.	2
GET /api/bills	List bills and filter by period/property.	2
POST /api/readings	Create a manual meter reading.	2
GET /api/readings	List readings for a meter/date range.	2
GET /api/reconciliation/:billId	Compare bill with valid same-period manual intervals.	3
GET /api/analytics/summary	Return verified metrics and trend series.	4
POST /internal/analytics/summary	Express-to-FastAPI request for tested Python summary.	6
POST /api/anomalies/detect	Run transparent detector and save result/method version.	7
POST /api/forecasts/generate	Generate and evaluate a forecast with caveats.	8
POST /api/insights/explain	Generate an evidence-grounded explanation from verified facts.	9

8. Suggested repository layout

```
wattwise/
■■■ frontend/ # React + Vite UI
■■■ backend/ # Express API, DB access, validation
■■■ analytics/ # Python, pandas, FastAPI, tests
```

```
■■■ database/ # migrations, schema, seed data
■■■ docs/ # scope, architecture, decisions
■■■ .env.example
■■■ README.md
```

Keep the project understandable. Avoid sharing code across Node and Python through copy-pasted logic; define API contracts and test the same known examples on both sides.

9. Working rhythm and completion criteria

- Work in small vertical slices: database → API → UI → tests for one feature, then move on.
- For every new Python/AI concept, write a tiny practice task and then apply it to WattWise.
- Track tasks in the included spreadsheet. Use Blocked when waiting on a decision/dependency, and add a note explaining the blocker.
- Before adding a model, first build a baseline and a test dataset. Before adding an LLM, first make sure all facts are computed by code.
- A strong finish is a reliable, explainable demo with limitations documented—not the largest number of AI buzzwords.

Recommended first milestone

Set up the repository, write the initial schema for bills and manual readings, and create test fixtures for matching, discrepant, and incomplete data. Do not start with the AI chatbot or ML model. Once this foundation is solid, the AI work will be much easier to trust and explain.